



# What's the Word?

A design-and-setup review: architecture, stack, CI, and a running prototype.

Mike · Josh · Alan · Razin

M1 → M2

## What changed, and the evidence behind it

| M1 assumption        | M2 decision               | Evidence                                                                               |
|----------------------|---------------------------|----------------------------------------------------------------------------------------|
| Yes / No only        | Yes / Maybe / No          | "Fine with it" was being counted as a reject — a neutral answer shouldn't sink a place |
| Gesture-only swipe   | Gesture + labeled buttons | Gestures alone weren't discoverable to first-time users                                |
| Forced top-3 results | Preserve ties in Results  | A real test run produced a 4-way tie for 1st place                                     |
| One sitting assumed  | Progress counter + resume | Participants get interrupted mid-Pick and need to pick up where they left off          |

Each row shipped as a code change, not a guess: the ranking engine, the swipe UI, and the resume logic.

## STATUS

# What works today

### RUNNING NOW

- ✓ Email-code authentication
- ✓ Seeded Pick, real Stony Brook places
- ✓ Private swipe answers (Yes/Maybe/No)
- ✓ Persistence: resumes after closing the app
- ✓ Server-side ranking (host taps Rank)
- ✓ Results: rank only, no scores shown
- ✓ Row-level security on every read
- ✓ CI: lint, types, tests, Android build

### NEXT (PLANNED)

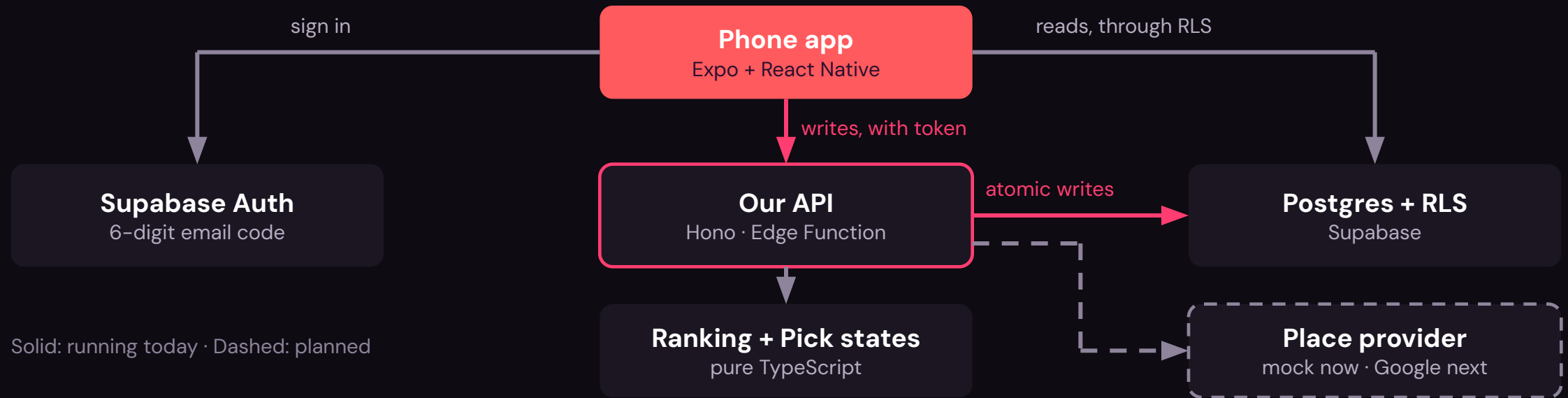
- ⌋ Create a Pick + invite by join code
- ⌋ Real place data via Google Places
- ⌋ Final vote among finalists
- ⌋ Live multi-device coordination

#### Provider note

Place data is mocked today by design — 20 real Stony Brook locations, normalized behind the same provider interface Google Places will use. Swapping providers touches one file, not the app.

## ARCHITECTURE

# Reads go through RLS; writes go through our API



**The rule this diagram enforces:** the app may read data it's permitted to see through RLS, but every sensitive write goes through our API. There is exactly one writer, so every rule about who can do what lives in code we unit-test — not scattered across screens or policies. Expanded in docs/design.md.

## TECH STACK

# Why this stack: choice, alternative, tradeoff

### Expo + React Native

**WHY:** One codebase for iOS and Android with native gestures.

**ALT:** Separate Swift + Kotlin, or a web app.

**TRADEOFF:** Faster to build; less low-level native control.

### TypeScript

**WHY:** One language — shared types from app to API to tests.

**ALT:** Plain JavaScript on both sides.

**TRADEOFF:** Still need runtime validation at the API boundary.

### Supabase (Postgres + Auth + RLS)

**WHY:** A relational model fits Picks, participants, candidates and preferences.

**ALT:** Firebase, or a fully custom backend.

**TRADEOFF:** Faster infra now; some platform coupling later.

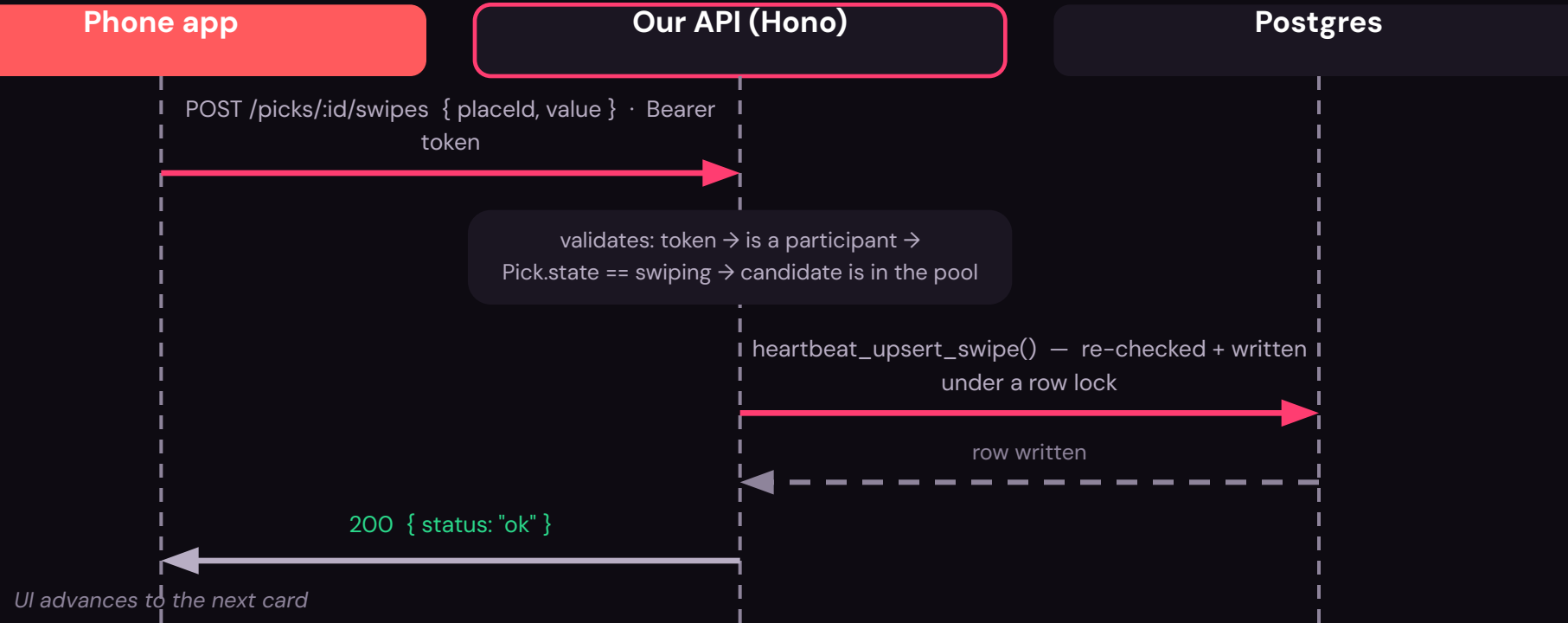
### Hono + Edge Function (our API)

**WHY:** One place for writes, auth checks, ranking and provider keys.

**ALT:** Logic spread across client code and DB policies.

**TRADEOFF:** One extra network hop; far easier to test and secure.

# One swipe through the system



## Invalid case

If the Pick already moved past Swiping (e.g. into Ranking), the same request instead gets HTTP 409, code `INVALID_PICK_STATE` — “Pick must be swiping.”

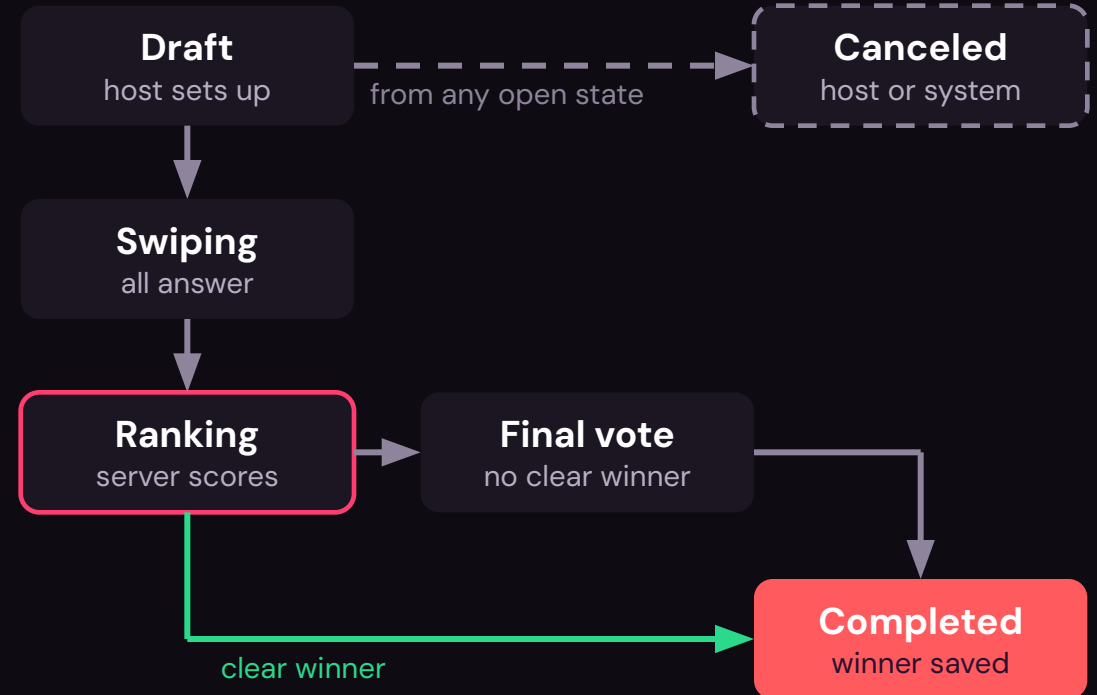
# Everything hangs off a Pick

## CORE TABLES



**Unique per row:** one participant per user per Pick · one candidate per place per Pick · one preference per user + candidate.

## PICK STATES



**Checked twice:** once in the API, again inside a locked database function, so a late swipe gets HTTP 409.

# Your answers are never shown to anyone



## Account required

Everyone in a Pick signs in with an email code. No guest links, no passwords stored.



## The app can only read

Row-level security filters every read. Only our API writes, with a key that never ships in the app.



## Your answers stay yours

You can read only your own answers. Scores never leave the server; the app shows rank only.



**We tested it:** 39 database tests and a 27-check end-to-end run. Friends' answers: **hidden**. Score column: **denied**. Direct writes from the app: **rejected**. Provider keys live only as server-side secrets, never in the app bundle.



## TESTING AND CI

# We test each layer for a different kind of failure

Unit · Jest

**121**

- ranking + tie-break math
- Pick-state transitions
- edge cases: missing answers, all No

API

**5**

- authenticated vs. unauthorized
- wrong-state request → 409
- response shape matches contract

Database · security

**39**

- RLS policy per table
- one user can't read another's answers
- direct client writes rejected

End-to-end

**27**

- sign in → swipe → close/reopen
- resume at the right card
- rank → results show ranks, not scores

EVERY PULL REQUEST RUNS THIS



Pull requests do not merge until CI is green and a teammate has reviewed them.

## DEMO

# Five actions, five things they prove



### 1. Sign in

*Proves: Real Supabase Auth and a persisted session, not a stub.*



### 2. Swipe a few places

*Proves: An authenticated API write, saved as a private preference.*



### 3. Close the app, reopen it

*Proves: State and session actually persist — resumes where you left off.*



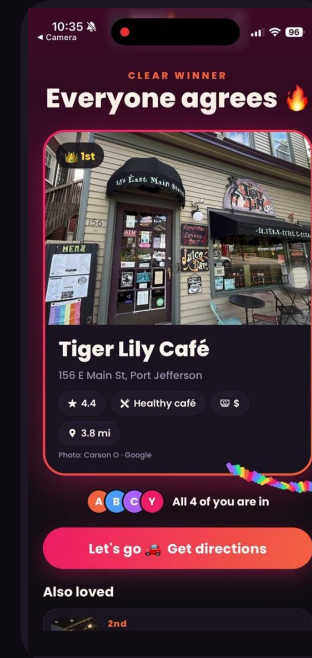
### 4. Finish, then trigger Rank

*Proves: The ranking formula runs server-side, against persisted answers.*



### 5. Show results

*Proves: The client receives a rank only — never raw private votes or scores.*



Recording

# What broke, and what we learned



## Results assumed a clean top 3

**Cause:** A real test run produced a 4-way tie for first place.

**Fix:** Results now shows everyone tied for #1, even past three.



## Reopening a Pick sometimes bounced you to Home

**Cause:** Screens rendered before the saved session finished loading.

**Fix:** The app now waits for the session before routing anywhere.



## The close-Pick logic assumed every Pick has a deadline

**Cause:** The database schema allows a null deadline.

**Fix:** Caught in review; “no deadline” now means only the threshold closes it, with a test.



## Our first API client didn't match the real API

**Cause:** The client and the API were built from one written contract, by different people, before either existed.

**Fix:** Checked side by side once both landed; fixed the mismatches and added a contract test.

## From a prototype to the full loop

### Create & invite

Start a Pick, set the area, share a join code.

### Final vote

One vote each among the finalists, then one saved winner.

### Real places

Google Places behind the same provider interface.

### Live coordination

Realtime progress across devices, no leaked answers.

# Questions?